

Internet of Things Connected Devices & Intelligent Systems

Embedded Systems, Protocols, Edge Computing, Cloud Integration & Security

Chapter 1: Understanding the IoT Ecosystem

The Internet of Things (IoT) refers to the network of physical devices — sensors, actuators, appliances, vehicles, and industrial equipment — embedded with electronics, software, and connectivity that enables them to collect and exchange data. By 2030, there will be over 29 billion connected IoT devices globally, generating unprecedented amounts of data about the physical world.

The four-layer IoT architecture:

Layer 1 — Perception	Physical sensors and actuators: temperature, humidity, accelerometers, cameras, GPS. These are the eyes and hands of IoT.
Layer 2 — Network	Communication protocols (MQTT, CoAP, LoRaWAN, Zigbee, BLE, Wi-Fi, 5G) that carry data from devices to gateways and the cloud.
Layer 3 — Processing	Edge gateways and fog nodes pre-process data locally before sending to the cloud — reducing latency, bandwidth, and cost.
Layer 4 — Application	Cloud platforms (AWS IoT, Azure IoT Hub, Google Cloud IoT), dashboards, analytics engines, and the business logic that creates value from data.

Chapter 2: Microcontrollers & Embedded Programming

IoT devices are powered by microcontrollers (MCUs) — small, low-power integrated circuits containing a CPU, RAM, flash memory, and peripherals on a single chip. Unlike a full computer, an MCU runs a single program (firmware) directly on hardware with no operating system overhead.

Popular IoT platforms:

Arduino (AVR / ARM)	Beginner-friendly, C/C++. 5V GPIO, abundant shields and libraries. Ideal for prototyping.
---------------------	---

ESP32	Dual-core 240 MHz, built-in Wi-Fi & Bluetooth, deep sleep modes for battery operation. Price: ~\$4. Workhorse of DIY IoT.
Raspberry Pi	Full Linux SBC. Runs Python, Docker, databases. Used as edge gateways or for vision/ML workloads at the edge.
STM32	Professional ARM Cortex-M MCUs with real-time capabilities. Used in industrial and automotive IoT.
MicroPython / CircuitPython	Run Python on ESP32/RP2040. Rapid prototyping with familiar syntax, smaller binary footprint than full CPython.

```
# MicroPython on ESP32 – read DHT22 sensor & control LED
import machine, dht, time, network, ujson
from umqtt.simple import MQTTClient

# Hardware setup
sensor = dht.DHT22(machine.Pin(4))
led     = machine.Pin(2, machine.Pin.OUT)

# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect('MyNetwork', 'MyPassword')
while not wlan.isconnected():
    time.sleep(0.5)
print('Connected:', wlan.ifconfig()[0])

# MQTT client
client = MQTTClient('esp32_001', 'broker.hivemq.com', port=1883)
client.connect()

while True:
    sensor.measure()
    payload = ujson.dumps({
        'temperature': sensor.temperature(),
        'humidity':    sensor.humidity(),
        'device_id':   'esp32_001'
    })
    client.publish(b'iot/home/climate', payload.encode())
    led.value(1); time.sleep(0.1); led.value(0) # blink heartbeat
    time.sleep(30) # publish every 30 seconds
```

Chapter 3: IoT Communication Protocols

Choosing the right protocol requires balancing range, power consumption, bandwidth, and cost. No single protocol fits every use case:

MQTT	Lightweight pub/sub protocol over TCP. QoS levels 0/1/2. Default IoT protocol for cloud connectivity. Port 1883 (8883 for TLS).
CoAP	Constrained Application Protocol — REST-like over UDP. Designed for very low-power sensors. Used in smart energy meters.
LoRaWAN	Long Range WAN — up to 15 km range, sub-mA sleep current, very low bandwidth (250 bps–50 kbps). Perfect for battery-powered sensors in agriculture and smart cities.
Zigbee	IEEE 802.15.4 mesh network. Short range (~10 m), self-healing mesh topology. Dominates smart home (Philips Hue, door sensors).
BLE 5.x	Bluetooth Low Energy — 10–40 m, coin-cell battery life for months. Used in wearables, asset tracking, proximity sensing.
5G / NB-IoT	Cellular IoT — NB-IoT for low-power wide-area; 5G for high-bandwidth applications like video surveillance and autonomous vehicles.

Chapter 4: Cloud Integration with AWS IoT Core

AWS IoT Core is a managed cloud service that lets billions of devices connect securely and exchange messages with AWS services. It handles connection management, device authentication via X.509 certificates, and message routing through a flexible rules engine:

```

# Python device SDK – publish sensor data to AWS IoT Core
import json, time, random
from awscrt import mqtt
from awsiot import mqtt_connection_builder

connection = mqtt_connection_builder.mtls_from_path(
    endpoint      = "xxxxxx-ats.iot.us-east-1.amazonaws.com",
    cert_filepath = "device.pem.crt",
    pri_key_filepath = "private.pem.key",
    ca_filepath    = "AmazonRootCA1.pem",
    client_id      = "factory_sensor_001",
    clean_session  = False,
)
connection.connect().result()

def publish_reading():
    payload = {
        "device_id": "factory_sensor_001",
        "timestamp": int(time.time()),
        "temperature": round(random.uniform(65, 85), 2),
        "vibration": round(random.uniform(0.1, 2.5), 3),
        "status": "normal"
    }
    connection.publish(
        topic      = "factory/machines/press_01/telemetry",
        payload     = json.dumps(payload),
        qos         = mqtt.QoS.AT_LEAST_ONCE
    )
    print(f"Published: {payload}")

# AWS IoT Rules Engine can route this data to:
# DynamoDB (store), Lambda (process), S3 (archive),
# SNS (alert), Kinesis (stream), SageMaker (ML inference)

```

Chapter 5: Edge Computing & IoT Security

Edge computing processes data on or near the device, rather than sending raw data to the cloud. This reduces latency (critical for real-time control), saves bandwidth costs, and enables offline operation. AWS Greengrass and Azure IoT Edge allow you to deploy cloud logic — including ML inference — directly onto edge devices.

IoT Security Imperatives:

- **Device Identity:** Every device must have a unique, cryptographically verifiable identity (X.509 certificate or TPM chip). Never use shared credentials.
- **Encrypted Communication:** All data in transit must use TLS 1.2+. Many IoT breaches exploit unencrypted MQTT on port 1883.

- **Secure Boot:** Verify firmware integrity at boot using a hardware root of trust. Prevents running tampered firmware.
- **OTA Updates:** Devices must support secure over-the-air firmware updates. Unpatched firmware is the #1 IoT vulnerability.
- **Network Segmentation:** IoT devices should be on an isolated VLAN, unable to reach corporate systems if compromised.

Real-World Attack: The Mirai botnet (2016) compromised 600,000+ IoT devices (cameras, routers, DVRs) by scanning for devices with default or weak credentials over Telnet. The resulting DDoS attack took down major internet services including Twitter, Netflix, and Reddit. Change default passwords — always.